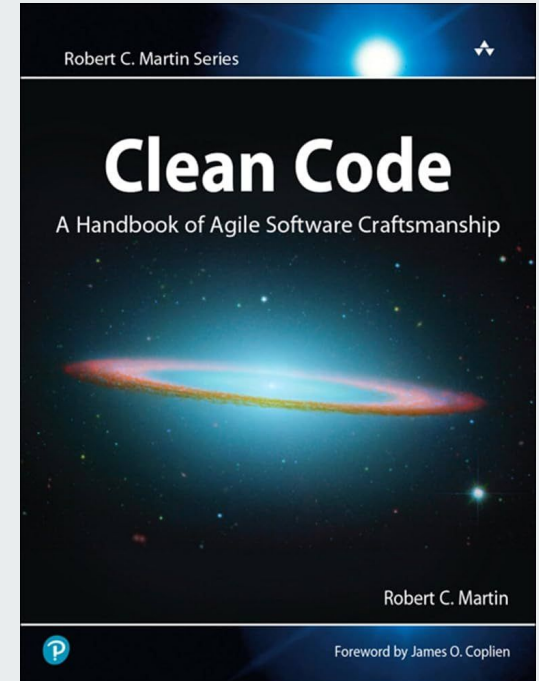




Clean Code

COMS 4156



Contents



- Clean Code
- General rules in Clean Coding
- Meaningful Names
- Functions
- Comments
- Formatting
- Data Abstraction
- Polymorphism
- Boundary Condition
- Unit Tests
- Examples
- Resources

What is Clean Code?

Elegance

I like my code to be
elegant and efficient

Clean code does one
thing well

Bjarne Stroustrup



Simple, direct, prose

Clean code is simple
and direct

Clean code reads like
well-written prose

Grady Booch



Small, expressive, simple

Reduced duplication,
high expressiveness,
and early building of
simple abstractions

Ron Jeffries



What you expected

You know you are
working on clean code
when each routine you
reads turns out to be
pretty much what you
expected

Ward Cunningham



Code is clean if it can be understood easily. Clean code can be read and enhanced by a developer other than its original author. With understandability comes readability, changeability, extensibility, and maintainability.

General Rules



The Boy Scout Rule

Leave the campground cleaner than you found it.

It's not enough to write the code well.

The code has to be kept clean over time. We have all seen code rot and degrade as time passes.

So we must take an active role in preventing this degradation.

Follow Standard Conventions

Coding Standard based on common industry norms.

How to name classes, methods, and variables; Where to put braces; and so on

Code should provide examples, so no document is needed.

Meaningful Names



- **Intention-Revealing Names** (The name should tell you why it exists, what it does, and how it is used.)
- **Avoid Disinformation** (Avoid words whose actual meaning varies from our intended meaning).
- **Use Pronounceable Names** (If you can't pronounce it, you can't discuss it).
- **Avoid Mental Mapping** (Clarity is king! Readers should not need to mentally map variables to the actual concept).

Functions



- Small
- Do One Thing **ONLY**
- Descriptive names
- Function arguments
 - Zero/One
 - Two
 - Argument Objects
- Command Query Separation: Do / Answer

Comments

Why do we use comments?

We use comments to compensate for our failure to express the intent of our code.

Comments Do Not Make Up for Bad Code

`// don't comment bad code, rewrite it!`

Formatting

Code formatting is about communication, and communication is the professional developer's first order of business.

The Purpose of Formatting

```
// communication
```

The Newspaper Metaphor

```
// high-level -> details
```

Vertical Openness Between Concepts

```
// each blank line is a visual cue  
// that identifies a new and separate concept
```



Data Abstraction

Concrete Point

```
public class Point {  
    public double x;  
    public double y;  
}
```

Abstract Point

```
public interface Point {  
    double getX();  
    double getY();  
    void setCartesian(double x, double y);  
    double getR();  
    double getTheta();  
    void setPolar(double r, double theta);  
}
```

- Both represent the data of a point on the Cartesian plane. And yet one exposes its implementation and the other completely hides it.
- Hiding implementation is about abstractions!
- Expose abstract interfaces that allow users to manipulate the essence of the data without having to know its implementation.

The Law of Demeter

```
final String outputDir = ctxt.getOptions()  
                              .getScratchDir()  
                              .getAbsolutePath();
```

Talk to friends, not to strangers.

A module should not know about the innards of the objects it manipulates.

A method f of a class C should only call the methods of these:

- C
- An object created by f
- An object passed as an argument to f
- An object held in an instance variable of C

Prefer polymorphism to if/else or switch/case.

```
class Bird {  
    // ...  
    double getSpeed() {  
        switch (type) {  
            case EUROPEAN:  
                return getBaseSpeed();  
            case AFRICAN:  
                return getBaseSpeed() - getLoadFactor() * numberOfCoco  
            case NORWEGIAN_BLUE:  
                return (isNailed) ? 0 : getBaseSpeed(voltage);  
        }  
        throw new RuntimeException("Should be unreachable");  
    }  
}
```

```
abstract class Bird {  
    // ...  
    abstract double getSpeed();  
}  
  
class European extends Bird {  
    double getSpeed() {  
        return getBaseSpeed();  
    }  
}  
  
class African extends Bird {  
    double getSpeed() {  
        return getBaseSpeed() - getLoadFactor() * numberOfCoconuts  
    }  
}  
  
class NorwegianBlue extends Bird {  
    double getSpeed() {  
        return (isNailed) ? 0 : getBaseSpeed(voltage);  
    }  
}
```

- Consider polymorphism before using switch.
- *Tell-Don't-Ask* principle: Tell the object what it needs to do and let it decide.
- Removes duplicate code.
- Very easy to add subclasses without touching existing code.

Encapsulate Boundary Conditions

```
if(level + 1 < tags.length)
{
    parts = new Parse(body, tags, level + 1, offset + endTag);
    body = null;
}
```

```
int nextLevel = level + 1;
if(nextLevel < tags.length)
{
    parts = new Parse(body, tags, nextLevel, offset + endTag);
    body = null;
}
```

- Hard to keep track of.
- Put all the processing in one place and not scattered all over the code.

Unit Tests

- Tests keep our code flexible, maintainable, and reusable.
- If you have tests, you do not fear making changes to your code. So keep tests clean.
- **Readable** tests: clarity, simplicity, and density of expression. Get right to the point and use only the data types and functions that you truly need.
- **One assert per test**: Quick and easy to understand. Minimize the number of asserts.
- F.I.R.S.T.
 - **Fast**: Run quickly to run them frequently.
 - **Independent**: Not depend on one another.
 - **Repeatable**: Run in any environment (QA/Prod).
 - **Self-Validating**: Return a boolean output of either passing or failing.
 - **Timely**: Written in timely fashion before production code.



Examples

Good & Bad!

Meaningful Names



```
public List<int[]> getThem() {  
    List<int[]> list1 = new ArrayList<int[]>();  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```

```
public List<int[]> getFlaggedCells() {  
    List<int[]> flaggedCells = new ArrayList<int[]>();  
    for (int[] cell : gameBoard)  
        if (cell[STATUS_VALUE] == FLAGGED)  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```



Avoid Disinformation

- Do not refer to a grouping of accounts as an `accountList` unless it's actually a `List`

-

```
int a = 1;  
if (0 == 1)  
    a = 01;  
else  
    1 = 01;
```

Make Meaningful Distinction

- *prodInfo* vs *prodData*?
- `getAccount()`
- `getAccounts()`
- `getAccountInfo()`

Use Pronounceable Names

```
class DtaRcrd102 {  
    private Date genymdhms;  
    private Date modymdhms;  
    private final String pszqint = "102";  
    /* ... */  
};
```

```
class Customer {  
    private Date generationTimestamp;  
    private Date modificationTimestamp;  
    private final String recordId = "102";  
    /* ... */  
};
```

Use Searchable Names

```
for (int j = 0; j < 34; j++) {  
    s += (t[j] * 4) / 5;  
}
```

```
int realDaysPerIdealDay = 4;  
const int WORK_DAYS_PER_WEEK = 5;  
int sum = 0;  
for (int j = 0; j < NUMBER_OF_TASKS; j++) {  
    int realTaskDays = taskEstimate[j] *  
realDaysPerIdealDay;  
    int realTaskWeeks = (realdays / WORK_DAYS_PER_WEEK);  
    sum += realTaskWeeks;  
}
```

Good Comments

Explain Yourself in Code

```
// Check to see if the employee is eligible for full
benefits
if ((employee.flags & HOURLY_FLAG) &&
    (employee.age > 65))

if (employee.isEligibleForFullBenefits())
```

Legal Comments

```
// Copyright (C) 2011 by Osoco. All rights reserved.
// Released under the terms of the GNU General Public
License // version 2 or later.
```

Clarification

```
assertTrue(a.compareTo(b) == -1); // a < b
assertTrue(b.compareTo(a) == 1); // b > a
```

Good Comments

Explanation of Intent

```
//This is our best attempt to get a race condition
//by creating large number of threads.
for (int i = 0; i < 25000; i++) {
    WidgetBuilderThread widgetBuilderThread =
        new WidgetBuilderThread(widgetBuilder, text, failFlag);
    Thread thread = new Thread(widgetBuilderThread);
    thread.start();
}
```

Warning of Consequences

```
public static SimpleDateFormat makeStandardHttpDateFormat() {
    //SimpleDateFormat is not thread safe,
    //so we need to create each instance independently.
    SimpleDateFormat df = new SimpleDateFormat("dd MM yyyy");
    df.setTimeZone(TimeZone.getTimeZone("GMT"));
    return df;
}
```

Bad Comments

Redundant Comments

```
// Utility method that returns when this.closed is true.  
// Throws an exception if the timeout is reached.  
public synchronized void waitForClose  
    (final long timeoutMillis) throws Exception  
{  
    if(!closed) {  
        wait(timeoutMillis);  
        if(!closed)  
            throw new Exception("MockResponseSender  
                                could not be closed");  
    }  
}
```

Noise Comments

```
/**  
 * Default constructor.  
 */  
protected AnnualDateRule() { }  
  
/** The day of the month. */  
private int dayOfMonth;
```

Scary Noise

```
/** The name. */  
private String name;  
  
/** The version. */  
private String version;  
  
/** The licenceName. */  
private String licenceName;  
  
/** The version. */  
private String info;
```

Bad Comments

Don't Use a Comment When You Can Use a Function or a Variable

```
// does the module from the global list <mod> depend on the
// subsystem we are part of?
if (smodule.getDependSubsystems()
    .contains(subSysMod.getSubSystem()))

// this could be rephrased without the comment as
ArrayList moduleDependees = smodule.getDependSubsystems();
String ourSubSystem = subSysMod.getSubSystem();
if (moduleDependees.contains(ourSubSystem))
```

Nonlocal Information

```
/**
 * Port on which fitnessse would run. Defaults to <b>8082</b>.
 *
 * @param fitnesssePort
 */
public void setFitnesssePort(int fitnesssePort)
{
    this.fitnesssePort = fitnesssePort;
}
```

Bad Comments

Attributions and Bylines

```
/* Added by Rick */
```

Commented-Out Code

```
InputStreamResponse response = new InputStreamResponse();  
response.setBody(formatter.getResultStream(),  
formatter.getByteCount());  
// InputStream resultsStream = formatter.getResultStream();  
// StreamReader reader = new StreamReader(resultsStream);  
// response.setContent(reader.read(formatter.getByteCount()));
```

Formatting

Vertical Distance

```
// variables
// should be declared as close to their usage as possible

// instance variables
// should be declared at the top of the class

// dependent functions
// if one function calls another, they should be vertically
// close, and the caller should be above the called

// conceptual affinity
// certain bits of code want to be near other bits
```

Horizontal Openness and Density

```
private void measureLine(String line) {
    lineCount++;
    int lineSize = line.length();
    totalChars += lineSize;
    lineWidthHistogram.addLine(lineSize, lineCount);
    recordWidestLine(lineSize);
}
```

Formatting

Team Rules

```
// every programmer has his own favorite formatting rules  
// but if he works in a team  
// then the team rules
```

Objects and Data Structures

Train Wrecks

```
Options opts = ctxt.getOptions();  
File scratchDir = opts.getScratchDir();  
final String outputDir = scratchDir.getAbsolutePath();  
  
final String outputDir = ctxt.options.scratchDir.absolutePath;
```

Final String outputDir = ctxt.options.scratchDir.absolutePath

This issue would be a lot less confusing if data structures simply had public variables and no functions, whereas objects had private variables and public functions.

Procedural vs Polymorphic

New function perimeter
VS
New object triangle

```
public class Square {
    public Point topLeft;
    public double side;
}

public class Rectangle {
    public Point topLeft;
    public double height;
    public double width;
}

public class Circle {
    public Point center;
    public double radius;
}

public class Geometry {
    public final double PI = 3.141592653589793;

    public double area(Object shape) throws NoSuchShapeException {
        if (shape instanceof Square) {
            Square s = (Square)shape;
            return s.side * s.side;
        }
        else if (shape instanceof Rectangle) { Rectangle r = (Rectangle)shape;
            return r.height * r.width;
        }
        else if (shape instanceof Circle) {
            Circle c = (Circle)shape;
            return PI * c.radius * c.radius;
        }
        throw new NoSuchShapeException();
    }
}
```

```
public class Square implements Shape {
    private Point topLeft;
    private double side;
```

```
    public double area() {
        return side*side;
    }
}
```

```
public class Rectangle implements Shape {
    private Point topLeft;
    private double height;
    private double width;
```

```
    public double area() {
        return height * width;
    }
}
```

```
public class Circle implements Shape {
    private Point center;
    private double radius;
    public final double PI = 3.141592653589793;
```

```
    public double area() {
        return PI * radius * radius;
    }
}
```

Resources

- <https://github.com/jnguyen095/clean-code/blob/master/Clean.Code.A.Handbook.of.Agile.Software.Craftsmanship.pdf>
- <https://github.com/JuanCrg90/Clean-Code-Notes>
- <https://gist.github.com/wojtekl/73c6914cc446146b8b533c0988cf8d29>
- <https://github.com/abiodunjames/Awesome-Clean-Code-Resources>
- <https://jwigert.github.io/clean-code-resources/>
- <https://blog.codacy.com/what-is-clean-code>



Questions?